

CSE331

AUTOMATA AND COMPUTABILITY

Introduction to Turing Machines

Spring 2026

Today's learning goals

Sipser Ch 2, 3.1

- Identify sets of strings as regular, context-free, or neither.
- Relate key differences between DFA, NFA, PDA, Turing machines and computational power.
- Trace the computation of a Turing machine using its transition function and configurations.
- Determine when a Turing machine is a decider.

Informal intuition

There must be at least
one language that is not
context-free

Which specific language is not context-free?

- A. $\{ 0^n 1^m 0^n \mid m, n \geq 0 \}$
- B. $\{ 0^n 1^n 0^n \mid n \geq 0 \}$
- C. $\{ 0^n 1^{2n} \mid n \geq 0 \}$
- D. $\{ 0^n 1^{2m} \mid m, n \geq 0 \}$
- E. I don't know.

Examples of non-context-free languages

- $\{ a^n b^n c^n \mid 0 \leq n \}$ Sipser Ex 2.36
- $\{ a^i b^j c^k \mid 0 \leq i \leq j \leq k \}$ Sipser Ex 2.37
- $\{ w w \mid w \text{ is in } \{0,1\}^* \}$ Sipser Ex 2.38

To prove... Pumping lemma for CFLs (won't cover in class)

Closure ?

The class of regular languages is closed under

- Union
- Concatenation
- Star
- Complementation
- Intersection
- Difference
- Reversal

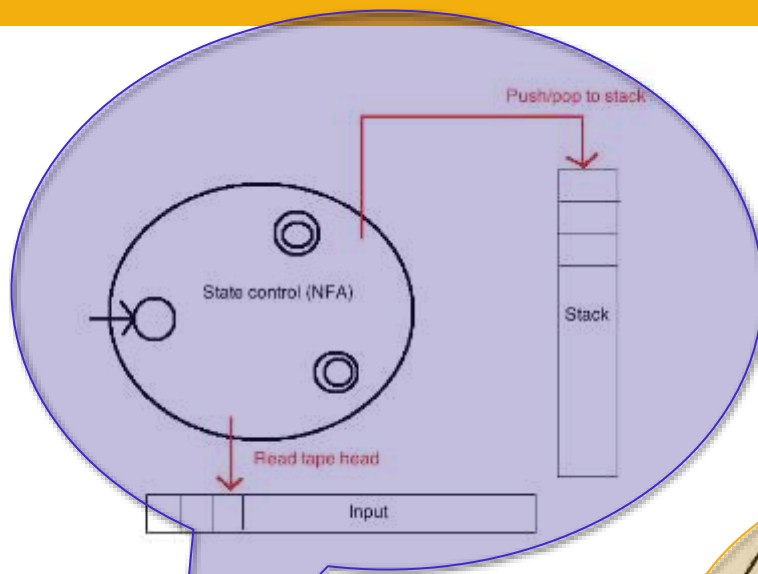
The class of context-free languages is closed under

- Union
- Concatenation
- Star
- Reversal

The class of context-free languages is not closed under

- Intersection
- Complement
- Difference

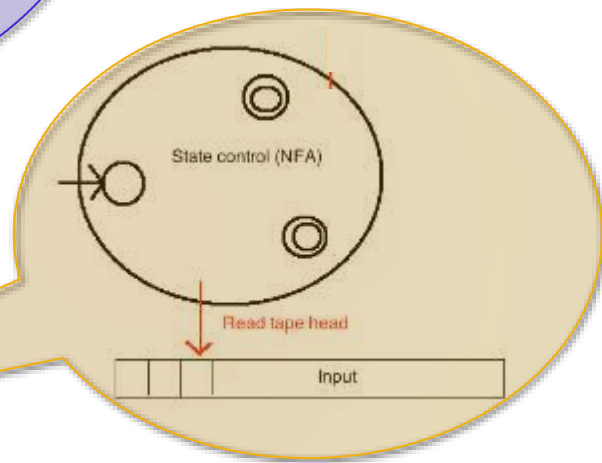
???



???

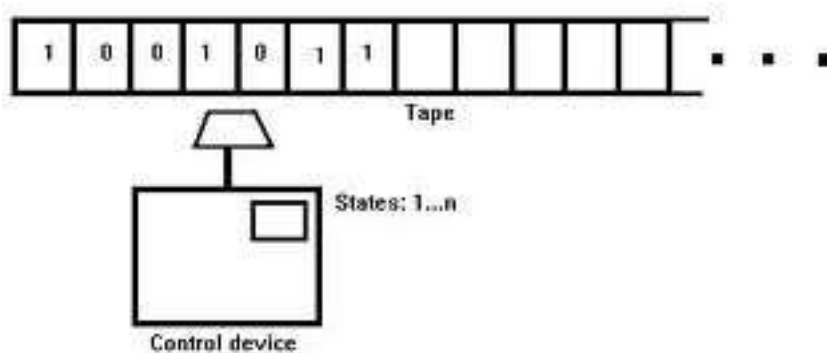
Context-free languages

Regular languages



Turing machines

- Unlimited input
- Unlimited (read/write) memory
- Unlimited time



The imitation game

- A movie about Turing and his role in breaking the Enigma code in WWII
- He made many inventions, one of them is the “Turing machine”



<https://www.youtube.com/watch?v=eWq5wAX8K8A>

Turing machine computation

- **Read/write head** starts at leftmost position on tape
- **Input string** written on leftmost squares of tape, rest is blank
- **Computation** proceeds according to transition function:
 - Given current state of machine, and current symbol being read
 - the machine
 - transitions to new state
 - writes a symbol to its current position (overwriting existing symbol)
 - moves the tape head L or R
- **Computation ends** if and when it enters either the **accept** or the **reject** state.

Language of a Turing machine

$L(M) = \{ w \mid \text{computation of } M \text{ on } w \text{ halts after entering the} \\ \textbf{accept state} \}$

i.e. $L(M) = \{ w \mid w \text{ is accepted by } M \}$

Comparing **TMs** and **PDAs**, which of the following is true:

- A. Both TMs and PDAs may accept a string before reading all of it.
- B. A TM may only read symbols, whereas a PDA may write to its stack.
- C. Both TMs and PDAs must read the string from left to right.
- D. States in a PDA must be either accepting or rejecting, but in a TM may be neither.
- E. I don't know.

Why is this model relevant?

- Division between program (CPU, state space) and data (memory) is a cornerstone of all modern computing
- Unbounded memory is outer limits of what modern computers (PCs, quantum computers, DNA computers) can implement.
- Simple enough to reason about (and diagonalize against), expressive enough to capture modern computation.

An example

$$L = \{ w\#w \mid w \text{ is in } \{0,1\}^* \}$$

We already know that L is

- not regular
- not context-free

We will prove that L is

the language of some Turing machine

An example

$$L = \{ w\#w \mid w \text{ is in } \{0,1\}^* \}$$

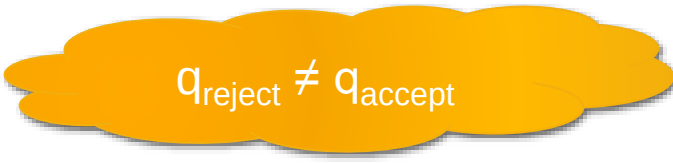
Idea for Turing machine

- Zig-zag across tape to corresponding positions on either side of '#' to check whether these positions agree. If they do not, or if there is no '#', *reject*. If they do, cross them off.
- Once all symbols to the left of the '#' are crossed off, check for any symbols to the right of '#': if there are any, *reject*; if there aren't, *accept*.

Formal definition of TM

A Turing machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where Q, Σ, Γ are all finite sets and

1. Q is the set of states
2. Σ is the input alphabet (not containing blank symbol)
3. Γ is the tape alphabet (including blank symbol as well as all symbols in Σ)
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function
5. $q_0 \in Q$ is the start state
6. $q_{\text{accept}} \in Q$ is the accept state
7. $q_{\text{reject}} \in Q$ is the reject state


$$q_{\text{reject}} \neq q_{\text{accept}}$$

Formal definition of TM

A Turing machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where Q, Σ, Γ are all finite sets and

1. Q is the set of states
2. Σ is the input alphabet (not containing blank symbol)
3. Γ is the tape alphabet (including blank symbol $\square$)
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function
5. $q_0 \in Q$ is the start state
6. $q_{\text{accept}} \in Q$ is the accept state
7. $q_{\text{reject}} \in Q$ is the reject state

Are **Turing machines** deterministic or not?

- A. Deterministic
- B. Nondeterministic
- C. Can be either
- D. I don't know

Configurations of a TM

- Current state
- Current tape contents
- Current location of read/write head

u q v

current state is q

current tape contents are uv (and then all blanks)

current head location is first symbol of v

Configurations of a TM

- Current state
- Current tape contents
- Current location of read/write head

$u q v$

current state is q

current tape contents are uv (and then all blanks)

current head location is first symbol of v

Start configuration on w :

$q_0 w$

Accepting configuration:

$u q_{acc} v$

Rejecting configuration:

$u q_{rej} v$

Halting configuration: any configuration that is either accepting or rejecting.

Transitioning between configurations



w is input, read/write head over the leftmost symbol of w

$$q' = \delta(q, v_1)$$

How does uv compare to $u'v'$?

Language of a TM

Sipser p. 144

$$L(M) = \{ w \mid M \text{ accepts } w \}$$

= { w | there is a sequence of configurations of M
where C_1 is start configuration of M on input w,
each C_i yields C_{i+1} and C_k is accepting configuration }

"The language of M"

"The language recognized by M"

Deciders and recognizers Sipser p. 144 Defs 3.5 and 3.6

- L is **Turing-recognizable** if some Turing machine recognizes it.
- M is a **decider** TM if it halts on all inputs.
- L is **Turing-decidable** if some Turing machine that is a decider recognizes it.

An example

$$L = \{ w\#w \mid w \text{ is in } \{0,1\}^* \}$$

We already know that L is

- not regular
- not context-free

We will prove that L is

~~the language of some Turing machine~~ Turing-decidable
(hence also Turing-recognizable)

An example

$$L = \{ w\#w \mid w \text{ is in } \{0,1\}^* \}$$

Idea for Turing machine

- Zig-zag across tape to corresponding positions on either side of '#' to check whether these positions agree. If they do not, or if there is no '#', *reject*. If they do, cross them off.
- Once all symbols to the left of the '#' are crossed off, check for any symbols to the right of '#': if there are any, *reject*; if there aren't, *accept*.

Is this machine a **decider**?

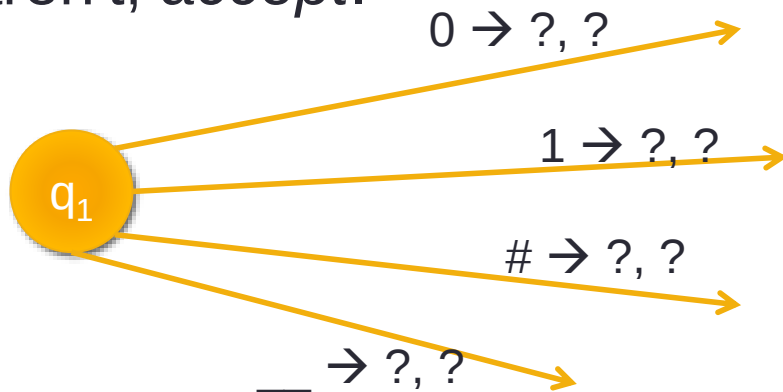
- A. Yes, because it reads the input string exactly once.
- B. Yes, because it will halt (and either accept or reject) no matter what the input is.
- C. No, because it sometimes rejects the input string.
- D. No, because it will go in an infinite loop if there's no '#'.
E. I don't know.

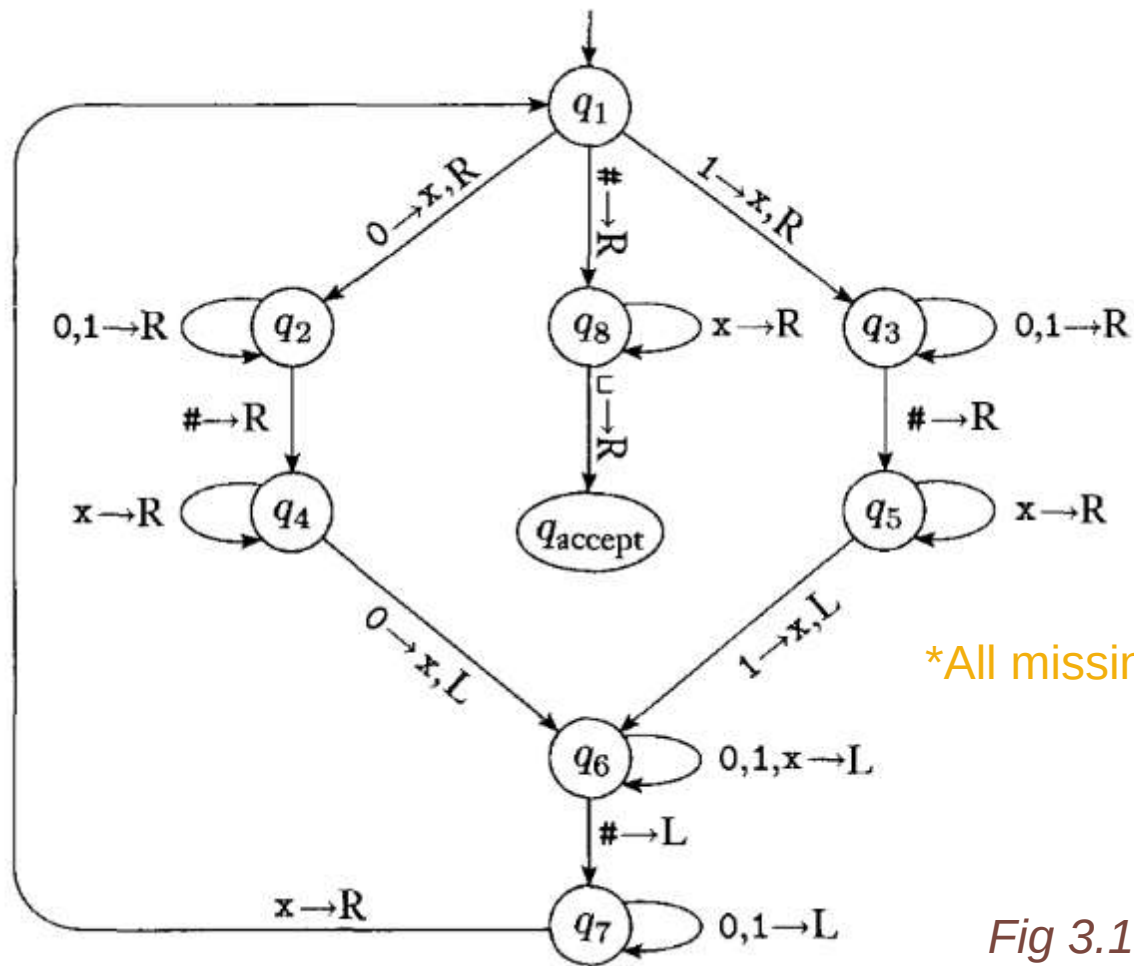
- Zig-zag across tape to corresponding positions on either side of '#' to check whether these positions agree. If they do not, or if there is no '#', *reject*. If they do, cross them off.
- Once all symbols to the left of the '#' are crossed off, check for any symbols to the right of '#': if there are any, *reject*; if there aren't, *accept*.

Idea for Turing machine

Zig-zag across tape to corresponding positions on either side of '#' to check whether these positions agree. If they do not, or if there is no '#', *reject*. If they do, cross them off.

Once all symbols to the left of the '#' are crossed off, check for any symbols to the right of '#': if there are any, *reject*; if there aren't, *accept*.





$Q =$

$\Sigma =$

$\Gamma =$

All missing edges go to reject state

Fig 3.10 in Sipser